ATIVIDADE INTEGRADORA | FASE 2

AURORA SIGER

Módulo de Gerenciamento de Pouso e Estabilização de Base

RELATÓRIO TÉCNICO COMPLETO

Lógica digital, algoritmos, modelagem matemática e ESG

Equipe

Integrante	RM
Gabriel Eduardo Alvite Mendes	576675
Beatriz Lopes de Lima	575983

Turma: 1CCOS

Repositório: github.com/Alvite01/missao-aurora-siger

2026

Sumário

Seção	Conteúdo	Página
1	Introdução e cenário de pouso	3
2	Regras de decisão com portas lógicas	4
3	Protótipo em Python	5
4	Modelagem matemática da descida	6
5	Contextualização histórica e arquitetural	7
6	ESG e governança	8
7	Conclusão	8
Anexo A	Estruturas de dados do MGPEB	9–10
Referências	Fontes bibliográficas e repositório	10

Nota de integridade do conteúdo

Os módulos, os parâmetros simulados, as variáveis lógicas e os resultados apresentados são provenientes do código mgpeb.py e do relatório técnico da equipe. O anexo integra esta edição, enquanto o código-fonte é entregue separadamente em formato .py. As limitações do protótipo são explicitadas nas seções pertinentes.

1. Introdução e cenário de pouso

O pouso é uma das etapas mais críticas de uma missão a Marte. O atraso de comunicação com a Terra, que pode ultrapassar vinte minutos, impede o controle direto em tempo real e exige decisões autônomas. As descidas dos rovers Curiosity e Perseverance utilizaram sequências automáticas; na missão Aurora Siger, o MGPEB gerencia a ordem de pouso dos módulos, autoriza ou bloqueia a operação e registra cada resultado.

O projeto integra portas lógicas, estruturas lineares, busca e ordenação em Python, funções matemáticas, arquitetura da computação e princípios de ESG. Os parâmetros são simulados e não representam telemetria de uma missão real.

1.1 Módulos de pouso definidos

Cinco módulos automatizados compõem a fila principal. Prioridade 1 é a mais alta, e a criticidade varia de 1 a 5.

ID	Módulo	Carga	Pri.	Comb.	Massa	Crit.	ETA
ENE-02	Energia	Painéis solares	1	48%	4.100 kg	5	10:05
HAB-01	Habitação	Habitat inflável	2	62%	5.200 kg	4	10:15
MED-05	Suporte médico	Equipamento médico	3	18%	2.800 kg	5	10:25
LAB-03	Laboratório científico	Sensores científicos	4	35%	3.300 kg	2	10:40
LOG-04	Logística	Equipamento de mineração	5	71%	6.000 kg	3	11:10

Tabela 1. Módulos de pouso e seus atributos (dados simulados).

A prioridade expressa dependências operacionais: o módulo de energia é processado primeiro porque sustenta os demais, seguido pelo habitat. A fila é ordenada por prioridade e, em caso de empate, criticidade. Os resultados são encaminhados às listas de módulos pousados, em espera ou em alerta; a pilha guarda os eventos e permite reverter a última ação. O detalhamento das estruturas está no Anexo A.

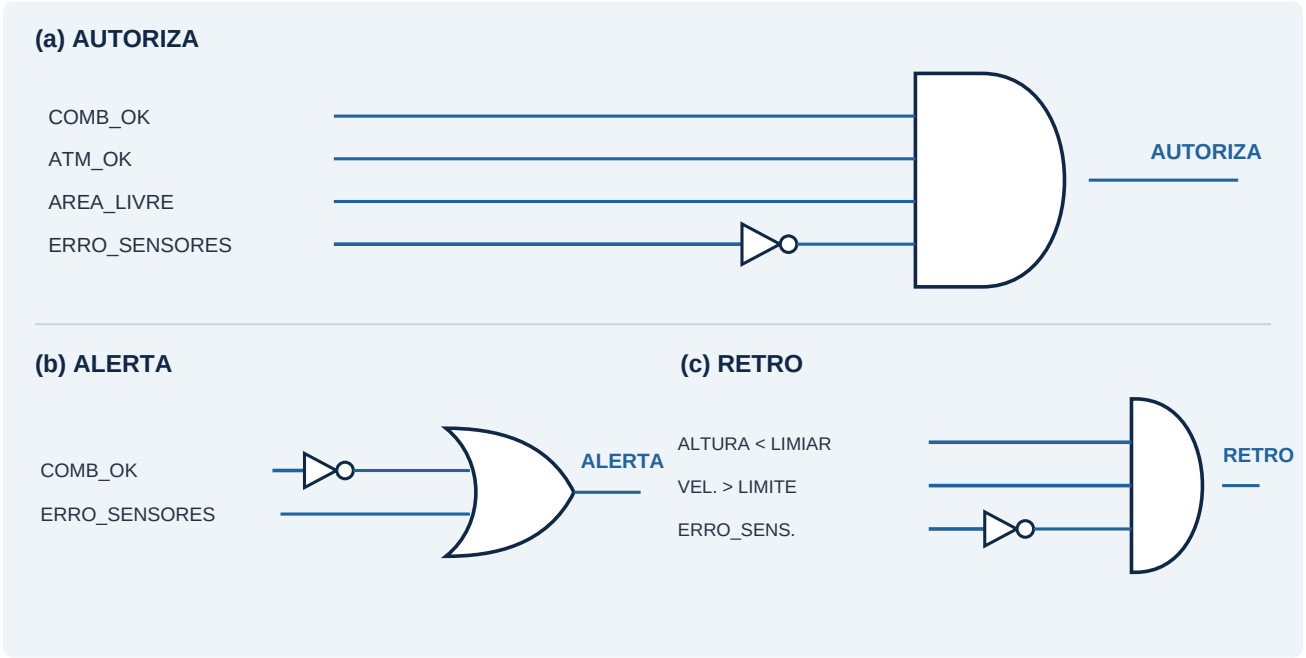
2. Regras de decisão com portas lógicas

A autorização de pouso é uma decisão binária baseada em quatro variáveis booleanas. A avaliação separa condições de segurança de situações que permitem aguardar nova tentativa.

Variável	Verdadeira quando...	Critério adotado
COMB_OK	Há combustível suficiente	Combustível ≥ 20%
ATM_OK	Condição atmosférica permite pouso	Vento ≤ 20 m/s; atual: 12 m/s
AREA_LIVRE	Zona de pouso desocupada	Entrada informada por módulo
ERRO_SENSORES	Falha na integridade dos sensores	Negação de sensores íntegros

Tabela 2. Variáveis booleanas do MGPEB.

```
AUTORIZA = COMB_OK AND ATM_OK AND AREA_LIVRE AND (NOT ERRO_SENSORES)
ALERTA = (NOT COMB_OK) OR ERRO_SENSORES
RETRO = (ALTURA < LIMIAR) AND (VELOCIDADE > LIMITE) AND (NOT ERRO_SENSORES)
```



Figuras 1 e 2. Circuitos completos com portas AND, OR e NOT convencionais.

A função AUTORIZA possui quatro entradas, com 16 combinações possíveis; somente a configuração COMB_OK = ATM_OK = AREA_LIVRE = 1 e ERRO_SENSORES = 0 permite o pouso. A lógica é conservadora: falhar em qualquer condição impede a autorização. ALERTA identifica combustível insuficiente ou sensores comprometidos; os demais bloqueios são classificados como adiamento.

2.1 Tabela-verdade e simplificação

ALT.<LIM.	VEL.>LIM.	ERRO	NOT ERRO	RETRO
0	0	0	1	0
0	0	1	0	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0

Tabela 3. Tabela-verdade da função RETRO.

Pelo teorema de De Morgan, NOT(ALERTA) = COMB_OK AND (NOT ERRO_SENSORES). Assim, AUTORIZA = NOT(ALERTA) AND ATM_OK AND AREA_LIVRE, permitindo reutilizar logicamente parte do circuito. No código, o fluxo segue if / elif / else: autorização, alerta e, por último, espera.

3. Protótipo em Python

O arquivo mgpeb.py utiliza dicionários, listas, funções e estruturas de controle básicas da linguagem, sem dependências externas. Seus sete blocos cobrem cadastro, estruturas lineares, buscas, ordenação, regras booleanas, modelo de descida e simulação principal.

Bloco	Funções ou operações principais	Complexidade
Estruturas	enfileirar; desenfileirar; empilhar; desempilhar	O(1); pop(0): O(n)
Busca	busca_menor; busca_linear; busca_binaria_prioridade	O(n); binária O(log n)
Ordenação	ordenacao_insercao; ordenacao_selecao	$O(n^2)$; inserção O(n) no melhor caso
Regras	pouso_autorizado; modulo_em_alerta; disparar_retro; decidir_pouso	O(1)
Modelo	altura_paraquedas; altura_retro; simular_descida	O(t/Δt)

Tabela 4. Organização e custos computacionais do protótipo.

Fluxo: cadastrar módulos; localizar combustível mínimo e criticidade máxima; ordenar por prioridade; aplicar as regras de pouso; mover os módulos para as respectivas listas; desfazer o último evento pela pilha; simular a descida. A busca binária ocorre após a ordenação.

```
ENE-02 -> POUSO AUTORIZADO | HAB-01 -> POUSO AUTORIZADO
MED-05 -> ALERTA (combustível 18%) | LAB-03 -> ALERTA (sensores)
LOG-04 -> ADIADO (área ocupada) | RETRO -> 16,5 s / 680 m
```

Os cenários exercitam os três ramos da decisão: dois pousos autorizados, dois alertas e um adiamento. Mesmo com criticidade máxima, o módulo médico é bloqueado por combustível insuficiente, preservando a segurança.

3.1 Testes de limites e comportamento seguro

A rotina testar_regras_limite(), executada ao final do programa, verifica automaticamente dez condições, incluindo valores exatamente nos limites e situações de bloqueio. Todos os dez testes passaram (10/10).

Condição testada	Resultado esperado
Combustível: 19% / 20%	Alerta / autorizado
Vento: 20 / 21 m/s	Autorizado / adiado
RETRO: altura 700 / 699 m	Não dispara / dispara (v=21 m/s)
RETRO: velocidade 20 / 21 m/s	Não dispara / dispara (h=699 m)
Sensor com erro; área ocupada	Alerta ou bloqueio, conforme a regra

Tabela 5. Testes funcionais de fronteira, todos aprovados.

4. Modelagem matemática da descida

A altura durante o pouso é modelada em três fases. O modelo determina o limiar de ignição dos retrofoguetes e permite avaliar como a velocidade e a capacidade de frenagem afetam a segurança.

```
FASE 1 - Paraquedas: h(t) = H0 - v0 . t
FASE 2 - Retrofoguetes: h(τ) = h1 - v0 . τ + (a/2) . τ²
v(τ) = v0 - a . τ
ALTURA MÍNIMA: hmin = (v0² - vf²) / (2a) = 639,6 m
FASE 3 - Descida final: h(t) = h2 - vf . t
```

Parâmetro	Significado	Valor
H0	Altura inicial ao abrir o paraquedas	2.000 m
v0	Velocidade de descida sob paraquedas	80 m/s
a	Desaceleração líquida dos retrofoguetes	5 m/s²
vf	Velocidade desejada no toque	2 m/s
LIMIAR	Altura de ativação, com margem sobre hmin	700 m

Tabela 6. Parâmetros simulados do modelo matemático.

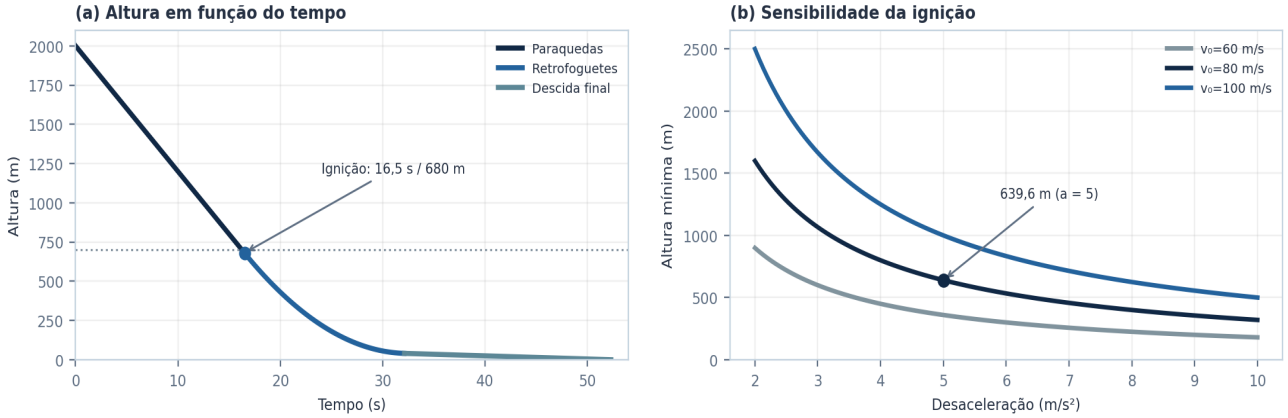


Figura 3. Altura × tempo e sensibilidade da altitude mínima de ignição.

4.1 Análise do gráfico

Na fase 1, a altura diminui linearmente à taxa de 80 m/s. Na fase 2, a desaceleração dos retrofoguetes reduz a velocidade de 80 m/s para 2 m/s, com altura final de 40,4 m; a fase 3 é uma descida linear lenta. A duração aproximada é de 52 s (16,5 + 15,6 + 20,2).

A altura mínima cresce com o quadrado da velocidade: a 100 m/s seriam necessários 999,6 m, em vez de 639,6 m a 80 m/s; a 60 m/s, 359,6 m. A maior desaceleração reduz essa altura: com a = 10 m/s², 319,8 m; com a = 3 m/s², 1.065,9 m.

4.2 Relação com as decisões do MGPEB

(i) O limiar de 700 m supera em aproximadamente 9% a altura mínima teórica de 639,6 m. Com passos de 0,5 s, o protótipo inicia a frenagem em t = 16,5 s, a 680 m. (ii) A variável AREA_LIVRE é informada por módulo como entrada, o que explica o adiamento do LOG-04; atualizá-la automaticamente após cada pouso seria uma evolução. (iii) Uma velocidade maior exige ignição mais alta; bloquear uma ignição atrasada seria uma verificação adicional ainda não implementada.

5. Contextualização histórica e arquitetural

A computação passou por máquinas mecânicas (Pascal e Babbage), eletromecânicas e eletrônicas a válvulas, incluindo o ENIAC, de 1945. A evolução para transistores, circuitos integrados e microprocessadores reduziu dimensões, consumo e custo. O Apollo Guidance Computer, nos anos 1960, operava com cerca de 2 mil palavras de RAM e 36 mil palavras de ROM. Computadores resistentes à radiação, como a família RAD750, priorizam confiabilidade em operações espaciais.

Em Marte, processamento, memória e energia são limitados; radiação pode afetar a memória; variações térmicas são extremas, e o atraso de comunicação exige autonomia. Assim, o MGPEB privilegia regras previsíveis e estruturas simples.

Limitação	Escolha adotada no MGPEB
Pouca memória e processamento	Fila, listas e pilha curtas; ordenação por inserção, in-place.
Energia limitada	Buscas simples em poucos elementos; binária após ordenação; regras O(1).
Radiação e erros de sensores	Regras determinísticas; evolução possível: votação redundante 2 de 3 (A·B + A·C + B·C).
Atraso de comunicação	Decisão autônoma; pilha de eventos com capacidade de reversão em memória.

Tabela 7. Restrições de hardware e justificativas do projeto.

O uso de Python é didático, pois evidencia a lógica e facilita a verificação. Em um sistema embarcado real, seriam necessários componentes e linguagens adequados a requisitos rigorosos de tempo, segurança e tolerância a falhas.

6. ESG e governança na base Aurora Siger

Os critérios ambientais, sociais e de governança (ESG), o triple bottom line de Elkington e o princípio de desenvolvimento sustentável do Relatório Brundtland são adaptados ao contexto da colônia. Em Marte, o resultado não é expresso apenas em lucro, mas em segurança, ciência e autonomia.

6.1 Critérios para escolha da área de pouso

Critério	Pilar	Justificativa
Proteção planetária e científica	E	Evitar regiões especiais e contaminação prejudicial; observar o Tratado do Espaço Exterior e diretrizes do COSPAR.
Segurança de terreno e ventos	S	Terreno plano, poucos obstáculos e ventos moderados diminuem riscos.
Acesso a recursos locais	E / G	Gelo de água e insolação reduzem dependência da Terra, com extração monitorada.
Distância do habitat	S / E	Separar pouso e habitação protege instalações de poeira e detritos.

Tabela 8. Critérios ambientais, sociais e de governança para o pouso.

6.2 Energia, resíduos e recursos locais

• Energia: geração solar e baterias; o módulo de energia tem prioridade 1 (ODS 7).

• Água: reciclagem em circuito fechado e extração mínima de gelo (ODS 6).

• Resíduos: reduzir, reutilizar e reciclar; destinação controlada (ODS 12).

• Recursos locais: uso da atmosfera e do solo, como produção de oxigênio a partir do CO_2 marciano, já demonstrada pelo experimento MOXIE no Perseverance (2021). Indicadores: energia consumida por habitante, percentual de água reciclada e massa de resíduos.

6.3 Governança, transparência e participação

Propõe-se um conselho de governança com representantes dos habitantes, da operação e da ciência, com regras claras para decisões sobre recursos e tecnologia (ODS 16). A pilha de eventos apoia a revisão das decisões, e relatórios periódicos com indicadores verificáveis no estilo GRI, complementados por auditoria independente, combatem o greenwashing. A participação dos habitantes pode ocorrer por consulta e voto, enquanto os princípios do Pacto Global orientam direitos humanos e condições dignas de trabalho na colônia (ODS 3 e 8). Alertas críticos, como o do módulo médico, requerem avaliação humana: a automação apoia, mas não substitui a responsabilidade das pessoas.

7. Conclusão

O MGPEB integra estruturas lineares, decisões booleanas, algoritmos, modelagem matemática e princípios de governança em uma simulação coerente de pouso. Como evoluções, recomendam-se a votação redundante dos sensores, condições atmosféricas mais realistas e a atualização automática da ocupação da área.

Anexo A - Estruturas de dados do MGPEB

Três estruturas lineares organizam o estado e as operações do projeto: a fila define o atendimento; as listas armazenam os resultados; e a pilha permite desfazer o último evento.



Figura 4. Fluxo dos módulos entre fila, regras lógicas, listas e pilha.

Estrutura	Variável no código	Operações e finalidade
Fila (FIFO)	fila_pouso	append e pop(0); cadastro, priorização e retirada em ordem.
Listas	lista_pousados; lista_espera; lista_alerta	append, percursos e buscas; identificam o estado dos módulos.
Pilha (LIFO)	pilha_eventos	append e pop; registra eventos e reverte a última ação.

Tabela 9. Estruturas lineares efetivamente usadas.

A.1 Exemplo concreto - fila

No cadastro, a fila segue a lista MODULOS_INICIAIS: [HAB-01, ENE-02, LAB-03, LOG-04, MED-05]. Esta é a ordem de registro no código, e não a sequência dos horários ETA. Após ordenação pela chave (prioridade, -criticidade), fica [ENE-02, HAB-01, MED-05, LAB-03, LOG-04]. A primeira remoção devolve ENE-02.

```
def enfileirar(fila, modulo): fila.append(modulo)
def desenfileirar(fila): return fila.pop(0)
```

A.2 Exemplo concreto - listas

Após processar a fila: pousados = [ENE-02, HAB-01]; em espera = [LOG-04]; em alerta = [MED-05, LAB-03]. A busca pelo menor combustível encontra MED-05 (18%), e a busca por criticidade 5 retorna ENE-02 e MED-05.

A.3 Exemplo concreto - pilha

Entrada	Evento	Destino	Pilha
1	ENE-02 - pouso autorizado	pousados	base
2	HAB-01 - pouso autorizado	pousados	
3	MED-05 - alerta	alerta	
4	LAB-03 - alerta	alerta	
5	LOG-04 - adiado	espera	topo

Tabela 10. Pilha de eventos após o processamento.

Ao executar desfazer_ultimo_evento, o registro de LOG-04 é removido do topo; o módulo sai da lista de espera e retorna ao início da fila para reavaliação. A pilha fornece histórico em memória, mas não substitui um registro permanente e imutável para auditoria.

```
def empilhar(pilha, evento): pilha.append(evento)
def desempilhar(pilha): return pilha.pop()
```

Desempenho. Em listas Python, pop(0) desloca os elementos e tem custo O(n). Para cinco módulos isso é suficiente. Para frotas maiores, um índice de início ou uma estrutura apropriada de fila pode reduzir o custo de remoção.

Referências

ELKINGTON, John. The Triple Bottom Line: Does It All Add Up? Pearson, 2018.
FLOYD, Thomas. Sistemas Digitais: Fundamentos e Aplicações. 11. ed. Pearson, 2016.
GOODRICH, Michael; TAMASSIA, Roberto; GOLDWASSER, Michael. Data Structures & Algorithms in Python. Pearson, 2016.
GUTTAG, John. Introduction to Computation and Programming Using Python. Pearson, 2021.
LAFORGE, Linda; WAITE, Sue. Mathematical Modeling and Applied Mathematics. Pearson, 2018.
STALLINGS, William. Arquitetura e Organização de Computadores. 10. ed. Pearson, 2021.
TANENBAUM, Andrew; AUSTIN, Todd. Estrutura e Organização de Computadores. 6. ed. Pearson, 2020.
COMISSÃO MUNDIAL SOBRE MEIO AMBIENTE E DESENVOLVIMENTO. Nosso Futuro Comum. ONU, 1987.
ORGANIZAÇÃO DAS NAÇÕES UNIDAS. Agenda 2030 (2015); Tratado do Espaço Exterior (1967); Pacto Global.
COSPAR. Policy on Planetary Protection. Committee on Space Research.

Materiais complementares

Código-fonte: mgpeb.py (arquivo de entrega). Repositório da missão, com o histórico do projeto e seus materiais: github.com/Alvite01/missao-aurora-siger. A modelagem apresentada constitui uma simulação didática; não representa um sistema certificado de voo.